

trailhead-agent

Project report: architecture, threat model, impact, novelties, research

Version 0.5.1 | Sources: README.md, DESIGN.md, src/trailhead_agent

Disclaimer: Engineering / product narrative. Not peer-reviewed research. Deep sections synthesize design docs and implementation intent.

1. Executive summary

What it is: A Python CLI that combines (A) **Playwright** discovery of Trailhead /content/learn/ unit links under strict URL policy, (B) **LLM ranking** with Pydantic-validated JSON and **href allowlisting** (no invented URLs), orchestrated by a mandatory **LangGraph** graph with optional planner and repair passes, and (C) optional **org** helpers (YAML checklists, sf visibility via CliOrgExecutor, org prepare --open-playground for browser + human Launch Playground).

Explicit non-goals: Auto-submit quizzes, credential stuffing, automatic Launch Playground clicks, crawling outside allowed Trailhead paths.

- **Observability:** trace_id, duration_ms, agent_version; --log-json; E2E JSON + .webm + e2e-report.html.
- **Demos:** Title-card phases (inputs, discovery, LLM order slide, next steps, walk); stderr narration (TRAILHEAD_DEMO_NARRATION).

2. Goals and problem fit

- **Assist** learners with **navigation and prioritization** using natural-language intent.
- **Do not** automate quizzes, knowledge checks, or credential abuse.
- **Separate concerns:** browser discovery vs. LLM reasoning vs. future Salesforce org automation.

3. Stakeholders and value

Learners	Intent-ordered queue (e.g. hands-on-first) vs. scanning long TOCs manually
Demos / L&D	Reproducible plan-primary vs org-prepare clips + JSON for narrative
Operators	Bounded URLs, exit codes, headless CI, JSON logs for pipelines
Extensibility	OrgExecutor protocol: noop today, cli/sf hooks without rewriting ranker

4. Layered architecture

Layer	Responsibility
CLI	Argparse; --json / --log-json; per-run trace_id in context; artifacts-dir sets TRAILHEAD_RECORD_VIDEO_DIR
Validation	validate_start_url(): HTTPS, TRAILHEAD_ALLOWED_HOST, path must include /content/learn/
Session	Chromium via Playwright; optional TRAILHEAD_BROWSER_USER_DATA_DIR + auth for SSO; optional email/password env login
Discovery	Scroll rounds, optional pagination clicks, nav wait policy; detect_discovery_blockers(); DiscoveryError with details (page URL/title) before LLM
LLM	llm_schemas + allowlist materialization; LangGraph: prepare → planner? → rank_primary → rank_repair? → finalize
Org	org_executor protocol; org_checklists.yaml; human-in-the-loop prepare and checklist commands

4.1 Logical component map

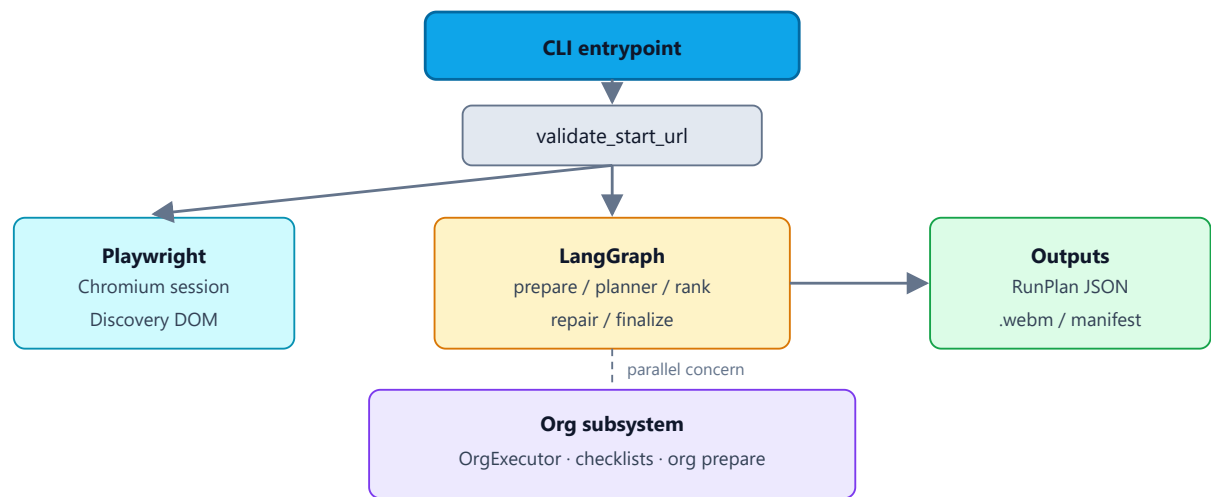


Figure 1 — Browser path feeds candidates; graph ranks; artifacts persist. Org is adjacent, not inside the ranker.

4.2 LangGraph ranking flow

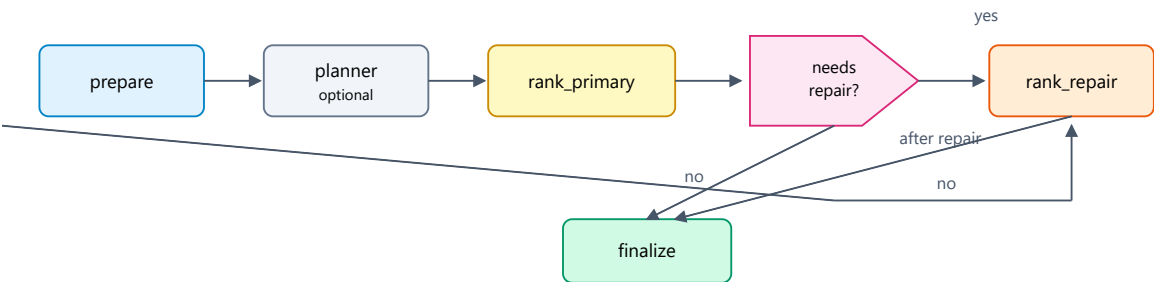
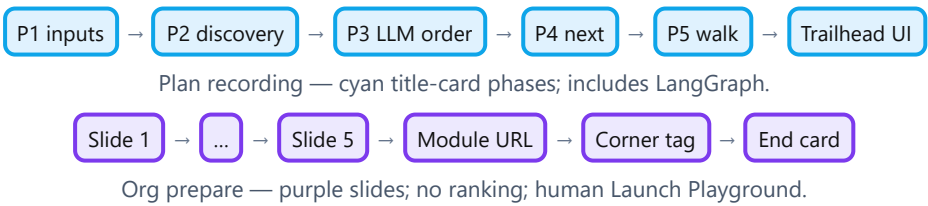


Figure 2 — Conditional repair loop; planner skipped unless LLM_PLANNER_PHASE=1.

4.3 Plan command vs org prepare



4.4 E2E artifact layout

ARTIFACTS_DIR/	
├ plan-primary.webm	main plan session video
├ org-prepare.webm	org prepare browser clip
├ open-unit-primary.webm	open-unit primary clip
├ plan-session-01.webm	optional sidecars
├ e2e-plan-<trace_id>.json	one plan run
├ e2e-plan-latest.json	symlink to latest semantics
├ e2e-open-unit-latest.json	
├ e2e-org-prepare-latest.json	org run summary + demo_guide
├ e2e-manifest.json	append-only recording index
└ e2e-report.html	gallery viewer

4.5 End-to-end data flow

```
start_url
-> Playwright: collect_units_for_seed -> List[UnitRef] (title, href; reason empty from scrape)
-> LangGraph.invoke: intent + candidates as dict rows
-> Optional planner notes merged into payload (LLM_PLANNER_PHASE=1)
-> Primary rank LLM call -> parse -> Pydantic validate -> materialize ordered hrefs subset
-> If invalid / empty allowlisted rows and repair enabled: repair LLM with excerpt + hints
-> finalize: emit result_units / result_hrefs; runner maps back to UnitRef with reasons
-> RunPlan JSON (+ optional walk_ranked navigations for recording)
```

5. LangGraph ranking

5.1 Nodes

```
prepare      Builds user JSON: intent, candidates[], ranking_user_message_context (execution_steps, hard_rules)
planner      If enabled: extra LLM notes only; skipped when LLM_PLANNER_PHASE != 1
rank_primary Single structured completion; ordered_units must use discovered hrefs only
rank_repair  If needs_repair and LLM_RANKING_REPAIR: second completion with invalid_output excerpt
finalize     Choose repair vs primary output; fallback to discovery order if nothing valid
```

5.2 Why a graph vs one shot

- **Repair path** is conditional: avoids paying for repair on happy path while recovering from schema drift or model mistakes.
- **Planner** is optional: adds latency/cost only when operators enable it.
- **Single orchestrator** (mandatory LangGraph) keeps ranking behavior consistent across providers.

6. Discovery subsystem

- **Lazy loading:** TRAILHEAD_DISCOVERY_SCROLL_ROUNDS merges anchors that appear after scroll.
- **Pagination:** TRAILHEAD_DISCOVERY_PAGINATION_CLICKS for Load more / Show more-style controls (bounded).
- **Navigation:** TRAILHEAD_DISCOVERY_NAV_WAIT_UNTIL (e.g. domcontentloaded vs load).
- **Failure semantics:** Zero usable units or degenerate TOC → DiscoveryError with hints; **no LLM call** (saves cost and avoids ranking garbage).
- **Blockers:** Login walls, geo, error pages can be flagged for operator action.

7. Org and Playground

- **plan** produces ranked JSON + optional browser walk on Trailhead only.
- **org prepare --open-playground** opens the module URL, title cards, dwell/scroll; **does not** run LangGraph ranking.
- **Launch Playground** remains a human action; TRAILHEAD_RECORDING_PLAYGROUND_CTA_SCROLL only scrolls CTAs into view for clearer recordings.
- **playground_ready** in JSON reflects executor/sf checks, not “browser clicked Launch.”
- **Checklists** map unit context to YAML steps (config/org_checklists.yaml) for hands-on guidance offline from the ranker.

8. Observability and E2E artifacts

Signal	Where
trace_id	contextvar; echoed in logs and plan JSON
duration_ms	Wall time for plan browser + LLM segment
video_files	Per-session clip list with role primary vs session; primary_video names main demo clip
e2e-manifest.json	Append-only timeline of plan / open-unit / org_prepare recordings in a directory
e2e-report.html	Self-contained gallery: videos, chips, manifest, JSON links, org panel

Sidecars: Persistent profiles closing extra tabs at session start may create short secondary .webm files; consumers should prefer primary_video.

8.1 Observability pipeline

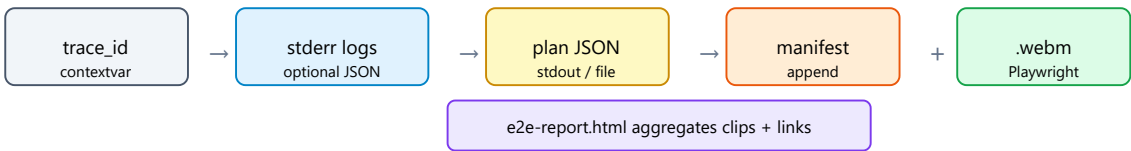


Figure 3 — One run ties logs, JSON, video, and HTML gallery under the same artifacts directory.

9. Threat model

Risk	Mitigation
Secret leakage	Keys in .env / CI only; not logged; .gitignore
Open redirect / SSRF	validate_start_url() host + path prefix
Prompt injection via page copy	Candidates are title/href structures; system prompt forbids new URLs
Academic dishonesty	No auto-answers; documented intent
Supply chain	Pinned minimum deps; CI tests
sf subprocess abuse	Argv lists; no shell=True; validated alias/paths

10. Failure modes

Failure	Behavior
DOM drift	Fewer links; tune waits, scroll, pagination, YAML selectors
SSO / MFA	Use auth + persistent profile
LLM 429 / quota	Tenacity retries (transport class); mapped user-facing errors
Bad LLM JSON	Repair pass or LLMPProviderError; empty ranking may fall back to discovery order
Windows SSL env	sanitize_environment() strips problematic values

11. Impact analysis

11.1 Intended positive impact

- **Learner throughput:** Converts “read entire TOC” into a single ranked list aligned to stated intent.
- **Interop:** JSON plan is stable input for checklists, internal LMS hooks, or human review.
- **Trust in demos:** Narration + title cards + manifest reduce “black box” perception of the agent.
- **Boundary clarity:** Explicit non-goals reduce misuse and support compliance conversations.

11.2 Limits and negative externalities

- **Vendor coupling:** Ranking quality and availability depend on LLM APIs and pricing.
- **Maintenance:** Trailhead UI changes are an ongoing tax on discovery, not a one-time parse.
- **No certified pedagogy:** Order is model-opinionated; not curriculum-authoritative without human review.

12. Novelties and differentiators

- **Hard grounding:** Output hrefs $\subseteq$ discovered hrefs — strong constraint vs free-form “suggested links.”
- **Graph-shaped LLM workflow:** Conditional repair and optional planner as first-class control flow.
- **Discovery as typed errors:** `DiscoveryError` with structured details and hints for operators.
- **Demo ergonomics:** Plan vs org visual language (cyan vs purple), epilogue navigation, friendly video filenames.
- **Org separation:** Ranking pipeline never implied to “complete labs”; prepare is explicitly adjacent.

13. Research positioning and evaluation gaps

Not claimed: New neural architectures, SOTA accuracy on a public Trailhead benchmark (none shipped in-repo).

Applied research adjacency: Constrained decoding / structured output; retrieval-grounded generation (candidates as retrieval set); policy-bound web agents; educational integrity.

13.1 What a rigorous evaluation would add

Gold set	Human expert ordering for N (start_url, intent) pairs frozen at a DOM snapshot
Metrics	Kendall tau / pairwise win rate vs gold; nDCG if partial relevance labels
Ablations	Planner on/off; repair on/off; model family swap; temperature
Cost curve	Tokens per plan; p95 latency (discovery vs LLM vs total)
Robustness	DOM snapshot replay vs live site drift (monthly re-run)

14. Ethics and misuse resistance

- Scope documentation and CLI boundaries reduce “take my quiz” expectations.
- Allowlisted hrefs limit exfiltration of arbitrary attacker-chosen URLs via model output (still does not eliminate all prompt risks).
- Human-in-the-loop org steps preserve accountability for org actions.

15. Roadmap and tradeoffs

15.1 Roadmap

1. Richer `OrgExecutor` (deploy, Tooling API, guided org browser).
2. Optional second-pass LLM checklist extraction from unit HTML (still HITL).
3. Disk cache keyed by (start_url, intent_hash).

4. Optional Pydantic Settings for env.

15.2 Tradeoffs

- **LLM vs rules:** Better intent flexibility vs token cost and uptime dependency.
- **Headless vs headed:** CI vs SSO realism.
- **Scope:** Full Trailhead+org automation would imply legal, ToS, and selector SLAs beyond this codebase.

16. Exit codes

```
0    Success
1    Generic / e.g. open-unit no navigation
2    Config, URL validation, DiscoveryError, OrgExecutorError (typical)
3    LLMPProviderError
130  Ctrl+C
```

17. Reference appendix

17.1 CLI surface

Command	Purpose
doctor	Version, LLM key presence, executor type, sf on PATH, optional org display JSON, recording dir hint
plan	Discovery + LangGraph rank; --json stdout; --walk-ranked N; --artifacts-dir
open-unit	Same ranking path; navigate first --visit-count ranked units
auth	Interactive sign-in into persistent Chromium profile
org doctor	sf / alias / executor diagnostics
org checklist	Markdown or JSON steps from plan file, single href, or live plan build
org prepare	Executor checks; optional --open-playground browser + optional --deploy

17.2 plan JSON shape

Emitted to e2e-plan-*.json and merged fields on stdout with plan --json. Saved bundle may add saved_at_utc, command, video_dir, video_files, etc.

Field	Type / meaning
agent_version	Package version string
label	Log label, often start URL
start_url	Validated module or trail URL
intent	User natural-language goal
units[]	title, href, reason per ranked row
trace_id	UUID per CLI invocation
duration_ms	Wall time browser + LLM segment
walk_ranked_visits	Count when --walk-ranked used
primary_video	Main .webm basename when recording
e2e_session_videos	All new session clips this run

17.3 LLM providers

LLM_PROVIDER	Typical env	Extra install
openai default	OPENAI_API_KEY, OPENAI_MODEL	base package
gemini	GOOGLE_API_KEY or GEMINI_API_KEY, GEMINI_MODEL	pip install -e "[gemini]"
anthropic	ANTHROPIC_API_KEY, ANTHROPIC_MODEL	pip install -e "[anthropic]"

Ranking knobs: LLM_PLANNER_PHASE=1 enables planner notes; LLM_RANKING_REPAIR=0 disables repair; LLM_OPENAI_STRICT_SCHEMA=1 OpenAI strict JSON schema mode.

17.4 Discovery environment

Variable	Role
TRAILHEAD_DISCOVERY_SCROLL_ROUNDS	Extra scroll passes for lazy anchors
TRAILHEAD_DISCOVERY_PAGINATION_CLICKS	Bounded clicks on load-more patterns
TRAILHEAD_DISCOVERY_NAV_WAIT_UNTIL	Playwright goto wait policy
TRAILHEAD_PAGE_SETTLE_MS	Post-navigation dwell
TRAILHEAD_MAX_MODULES	Caps trail crawl breadth

17.5 Recording and demo browser

Variable / flag	Effect
TRAILHEAD_RECORD_VIDEO_DIR / --artifacts-dir	Playwright saves .webm; enables e2e JSON writes
TRAILHEAD_RECORDING_DEMO_AUTO	Headed + slow_mo + demo viewport when recording
--no-recording-demo-browser	Sets demo auto off for CI-style runs
TRAILHEAD_DEMO_TITLECARDS	Full-screen phase slides in browser
TRAILHEAD_DEMO_TITLECARD_MS	Dwell per title card
TRAILHEAD_DEMO_TITLECARD_RANK_MAX	Max units listed on LLM order slide
TRAILHEAD_DEMO_NARRATION	Stderr step trace; off with =0
TRAILHEAD_PLAN_RECORDING_TAIL_MIN_MS	Dwell on live Trailhead after title cards
TRAILHEAD_ORG_PREPARE_RECORDING_MIN_MS	Minimum settle after org prepare navigation
TRAILHEAD_RECORDING_PLAYGROUND_CTA_SCROLL	Scroll Launch/Playground CTAs into view
TRAILHEAD_E2E_FRIENDLY_VIDEO_NAMES	Rename hash webm to plan-primary / org-prepare / etc.
TRAILHEAD_E2E_HTML_REPORT	Write e2e-report.html

17.6 Python dependencies

playwright, pyyaml, python-dotenv, openai, pydantic, tenacity,
langgraph, langchain-core
optional: google-genai, anthropic, google-generativeai legacy

17.7 Test and quality gates

pytest tests	unit + integration style harnesses
ruff / mypy	CI per pyproject / workflows
Golden fixtures	ranking JSON and plan shape tests

18. Conclusion

trailhead-agent is a **bounded assistant**: strong URL and output constraints, observable runs, and a deliberate split between “what to study next” (plan) and “how to open Trailhead / org” (prepare). Its deepest contributions are **systems integration**, **safety framing**, and **operator-grade artifacts**, not new core ML. Formal evidence would require gold rankings and measured ablations outside the current repository scope.

PDF: open this HTML in Edge/Chrome, Ctrl+P, Save as PDF. Or run `scripts\print_docs_to_pdf.ps1` (Windows).